

Causal-Branch Parameterization of Reidemeister-III Unlocking

A restricted quasi-polynomial simplification theorem and an integration-ready search for
ProveIt

OpenAI

Prepared for Vladimir Reshetnikov’s ProveIt repository

October 7, 2026

Reviewed baseline: commit 23893144e3aafd074a23cfb633222ab7e03e6d2e

Abstract

The maintained ProveIt unknot recognizer uses a fast exact Khovanov/Bar–Natan fallback and a crossing-nonincreasing Reidemeister preprocessor. Its current Reidemeister-III (RIII) helper searches short sequences, but after the first move it follows only the neighborhood of the most recent move. The repository records substantial empirical gains from this search, while explicitly leaving general quasi-polynomial unknot recognition open.

This report isolates and solves the parameterized combinatorial problem behind that helper. For a bounded-local reversible rewrite system, we define the *birth number* b of a first-unlocking trace to be the number of neutral actions whose footprints are disjoint from all earlier footprints. We prove: (i) the action-footprint graph of every shortest first-unlocking trace, including its terminal decreasing action, is connected; (ii) all birth actions can be commuted to the beginning; and (iii) traces of length at most k and birth number at most b can be enumerated in

$$N^{b+O(1)}(Ck)^k$$

time and polynomial working memory, where C depends only on the finite local rule set. In the fixed-dart model used by ProveIt, an RIII footprint has at most 24 darts and bounded local incidence, so the theorem applies with an absolute constant. Repeating the search between crossing-decreasing RI/RII phases gives a sound unknot preprocessor. It is quasi-polynomial on families with $b, k = O(\log N)$, improves to $N^{O(\log \log N)}$ for $b = O(1)$ and $k = O(\log N)$, and is polynomial for $b = O(1)$ and $k = O(\log N / \log \log N)$.

A proof audit also finds and repairs a tempting false claim: shortest local traces need not have connected chronological prefixes. Two independent preparatory branches may be joined only by a later move. The birth parameter is therefore necessary. The bundle contains a dependency-free reference implementation, strict counterexamples separating the current last-neighbor search from accumulated-support search, a finite oracle comparison on 5,000 generated local systems, and an opt-in ProveIt adapter targeting the reviewed `simplify.py` blob. No theorem that arbitrary unknot diagrams have logarithmic b or k is claimed, and the adapter has not been run through the full repository regression suite in this environment.

Contents

1 Executive summary

5

1.1	What is new	5
1.2	What this proves for unknot simplification	5
1.3	What this does not prove	6
1.4	Artifact map	6
2	Reviewed baseline, motivation, and scope	6
2.1	The maintained ProveIt pipeline	6
2.2	Relation to known Reidemeister bounds	7
2.3	Relation to the exact fallback	7
2.4	Methodological scope	7
3	A bounded-local reversible rewrite model	8
3.1	States, actions, and stable sites	8
3.2	Strong locality	8
3.3	Incidence bounds	9
3.4	Unlocking traces	9
3.5	The action-support graph	10
4	Causal structure and the birth-front normal form	10
4.1	Shortest traces have connected dependency graphs	10
4.2	Why connected prefixes are false	10
4.3	Births commute to the front	11
4.4	Accumulated support strictly improves last-touch search	12
5	Parameterized enumeration and asymptotic regimes	12
5.1	The birth-front search	12
5.2	Counting search nodes	13
5.3	Quasi-polynomial and polynomial corollaries	13
5.4	Search budgets	14
6	Specialization to Reidemeister-III unlocking	14
6.1	Fixed darts and the face permutation	14
6.2	A conservative RIII footprint	15
6.3	Disjoint footprints commute	15
6.4	Global and local incidence	16
6.5	Why accumulated footprints are the right implementation object	16
6.6	Interaction with the face shortcut	16

6.7	State restoration	17
7	The restricted recognizer and integration design	17
7.1	A monotone simplification class	17
7.2	Certificate size and replay	18
7.3	Opt-in API	18
7.4	Pseudocode	18
7.5	Required production tests	19
7.6	Why the exact fallback remains essential	19
8	Executable validation and negative controls	20
8.1	Reference implementation	20
8.2	Hand-checked strict examples	20
8.3	Generated finite systems	20
8.4	Exhaustive three-site catalog	21
8.5	Unit tests	21
8.6	What was not tested	22
8.7	Interpretation	22
9	Consequences, comparisons, and limitations	22
9.1	A new structural target	22
9.2	Why the result is stronger than a depth-only bound	23
9.3	Why polynomial Reidemeister length is not enough	23
9.4	Potential relation to planar separators	23
9.5	Potential relation to hierarchy certificates	23
9.6	Lower-bound questions	24
9.7	Soundness boundary	24
9.8	Assessment of the research contribution	24
10	Further research questions and proposed program	24
10.1	Immediate repository work	24
10.2	Parameterized topology	25
10.3	Algorithmic improvements to the birth exponent	25
10.4	Combination with exact invariants	26
10.5	Formal verification	26
10.6	A focused next milestone	26

A	Reproducibility, audit trail, and exact commands	27
A.1	Reviewed source pins	27
A.2	Bundle commands	27
A.3	Claims ledger	27
A.4	How to audit the key proof mechanically	28
A.5	PDF verification	28
A.6	Repository integration boundary	28

1 Executive summary

1.1 What is new

The principal new object is a causal parameter for local Reidemeister search. At a stalled diagram, an RIII move is *neutral*: it preserves crossing number but can change which RI/RII reductions are available. A sequence of neutral moves is *first-unlocking* when no RI/RII move is available before the last neutral move, and at least one is available immediately afterwards. Each RIII trial changes a bounded set of fixed darts. We call that set its *footprint*.

A move is a *birth* when its footprint is disjoint from the union of all previous footprints. The birth number b counts independent causal branches. All other moves touch the accumulated footprint. The core normal-form theorem says that every first-unlocking trace can be replaced, without increasing its length or birth number, by one in which all births occur first. This is not a mere presentation convention: it converts a global search into b global choices followed by a bounded-local search.

For depth k and birth budget b , the resulting deterministic search has the form

$$\underbrace{N \cdots N}_{b \text{ global births}} \underbrace{(Cb)(C(b+1)) \cdots (Ck)}_{\text{connected extensions}},$$

up to polynomial bookkeeping. Thus its running time is bounded by $N^{b+O(1)}(Ck)^k$.

1.2 What this proves for unknot simplification

Let D be an N -crossing diagram. Fix deterministic orders for RI/RII exhaustion and for the complete birth-front tree, and consider the crossing-nonincreasing scheme that repeatedly:

1. exhausts all available RI/RII reductions;
2. when stalled, searches for an RIII first-unlocking trace of depth at most k and birth number at most b ;
3. applies the trace and resumes RI/RII reduction.

Every successful run is a checkable Reidemeister certificate that D is an unknot. Completeness below is for the policy-defined class on which these fixed choices reach the empty diagram; mere existence of some different successful episode sequence would require outer backtracking and is not claimed. There are at most N successful unlock episodes, because every episode is followed by a reduction removing at least one crossing. Therefore the total running time remains

$$N^{b+O(1)}(Ck)^k.$$

This yields three clean regimes.

Birth bound	Depth bound	Consequence
$b = O(1)$	$k = O(\log N / \log \log N)$	polynomial time
$b = O(1)$	$k = O(\log N)$	$N^{O(\log \log N)}$
$b = O(\log N)$	$k = O(\log N)$	$N^{O(\log N)}$

The last two are quasi-polynomial. The first is a genuinely polynomial restricted class.

1.3 What this does not prove

No argument in this report shows that every unknot diagram admits a crossing-nonincreasing RIII/RI/RII simplification with logarithmic depth or birth number. General Reidemeister sequences may need crossing-increasing moves, and Lackenby’s polynomial upper bound on the total number of Reidemeister moves does not imply the monotone local hypotheses used here. The complete ProveIt recognizer must retain its exact fallback.

The result should therefore be read as a new explicit-parameter and quasi-polynomial *island*: it identifies two explicit structural parameters whose control is sufficient, supplies the complete search under those parameters, and narrows the remaining geometric problem to proving or predicting such control on important diagram families.

1.4 Artifact map

The accompanying archive contains:

- this article in LaTeX and PDF;
- `prototype/causal_search.py`, a generic executable model;
- `tests/test_causal_search.py`, including two strict separation fixtures;
- `experiments/exhaustive_validation.py` and the resulting JSON from 5,000 deterministic generated systems;
- `experiments/exhaustive_catalog.py`, covering a finite catalog of 608,400 three-site instances without sampling;
- `integration/clustered_r3_snippet.py`, an opt-in adapter for the private ProveIt `_Darts` representation;
- a review patch and explicit integration checklist;
- a claims ledger distinguishing proved, tested, untested, and open items.

2 Reviewed baseline, motivation, and scope

2.1 The maintained ProveIt pipeline

The reviewed source is the public ProveIt repository at commit `23893144e3aafd074a23cfb633222ab7e03e6d2e`. At that commit, the project combines cheap diagram reductions and obstructions with exact Khovanov/Bar–Natan computation. The commit message records 431 maintained tests and states explicitly that general quasi-polynomial recognition remains unproved. This report accepts that boundary [9].

The relevant preprocessor is in `Topology/UnknotRecognition/fast/fastunknot/simplify.py`, reviewed blob `2fcaabb51eaa45cf84bdee32cd1b4e18acf2c478`. The file represents a diagram by a fixed list of crossing darts and a mutable edge involution. It performs crossing-decreasing RI/RII moves incrementally. When no such move remains, it can trial an RIII inversion of a triangular face, keep a short sequence if the sequence exposes RI/RII, and otherwise undo it exactly.

The existing recursive helper has three important properties.

1. It is *sound*: successful RIII trials are retained only when a legal RI/RII move appears, and the resulting trace is independently replayable.
2. It is *budgeted*: the total number of trial moves is capped per input crossing, because failed deep searches can dominate runtime.
3. It is *path-local*: after the first RIII move, later candidates are drawn from a neighborhood of the immediately preceding move, rather than from the accumulated support of the entire trace.

The synthesis reports document why this matters. On a corpus of difficult unknot survivors, depth two, three, and four searches simplified many more instances than one-step lookahead, but a failed depth-four search on a triangle-rich braid was much more expensive. The current default therefore uses shallow depth and a strict budget. These data motivate a theorem that separates global branching from genuinely local continuation.

2.2 Relation to known Reidemeister bounds

Hass and Lagarias proved an exponential upper bound on the number of Reidemeister moves needed to trivialize an n -crossing unknot diagram [3]. Dynnikov proved monotonic simplification for arc-presentations of the unknot [2]; Henrich and Kauffman used that framework to bound diagram complexity and move counts [4]. Lackenby later proved that an n -crossing unknot diagram can be reduced using at most $(236n)^{11}$ Reidemeister moves while the crossing number remains at most $(7n)^2$ along the sequence [6].

Those theorems establish polynomial-size certificates and powerful global control. They do not assert that a sequence can be partitioned into bounded runs of RIII moves separated only by crossing-decreasing RI/RII moves. The present search deliberately forbids crossing increase. Consequently, the new parameterized theorem is neither a replacement for Lackenby’s theorem nor a consequence of it.

Lackenby’s 2026 hierarchy algorithm gives a different route to unknot detection, through incompressibility and essential hierarchies [7]. Its abstract emphasizes avoidance of certain large searches, but the preprint does not claim the quasi-polynomial running-time theorem sought by this project. The local RIII analysis here is orthogonal and can coexist with hierarchy-based certificates.

2.3 Relation to the exact fallback

Khovanov homology detects the unknot [5], and Bar–Natan’s tangle/cobordism formalism supports local scanning computations [1]. ProveIt exploits this exact route after cheaper stages fail. Report 25 changes none of that algebra. Its role is to make one preprocessing search asymptotically accountable and more complete under explicit parameters.

2.4 Methodological scope

The OpenAI mathematics repository contains manuscripts and proof artifacts at multiple verification levels, explicitly warning that unformalized outputs may contain errors [8]. This report follows the useful part of that model: theorem statements are separated from experiments, failed proof ideas

are recorded, and the integration patch remains opt-in. It does not rely on repository size or model provenance as mathematical evidence.

Warning 2.1 (Novelty and verification). The theorems below are new relative to the reviewed ProveIt source and its synthesis, but no claim of priority in the wider mathematical literature is made without expert review. The proofs are human-readable and computationally stress-tested, not formally verified. The ProveIt adapter was not executed inside a full checkout in this environment.

3 A bounded-local reversible rewrite model

This section isolates exactly the hypotheses used later. The model is broad enough for the fixed-dart Reidemeister implementation and narrow enough that commutation arguments are valid without hidden geometric assumptions.

3.1 States, actions, and stable sites

Fix a finite set V of *sites*, with $|V| \leq c_0 N$ for an input-size parameter N . A state belongs to a set Ω_N . There are two kinds of local operation.

Definition 3.1 (Neutral action). A neutral action instance a legal at $x \in \Omega_N$ is a reversible transition

$$x \xrightarrow{a} ax$$

with a footprint $\Phi_x(a) \subseteq V$. We assume $|\Phi_x(a)| \leq f$, where f is independent of N .

Definition 3.2 (Terminal reduction). A terminal reduction instance r legal at x is a local decreasing operation with footprint $\Psi_x(r) \subseteq V$, with $|\Psi_x(r)| \leq g$. During an unlocking search we only ask whether r is legal; the reduction is applied after a trace succeeds.

For RIII, the neutral action preserves the number of crossings. RI/RII are the terminal reductions. Stable sites are fixed input darts, so a crossing can be moved combinatorially without renaming its darts.

3.2 Strong locality

The main axiom is a two-sided commutation diamond. It is stated for executed action instances, because a later move need not be syntactically visible before an earlier local update.

Definition 3.3 (Strong disjoint commutation). A rewrite system has strong disjoint commutation when the following hold.

- (L1) **Backward and forward diamond.** Suppose $x \xrightarrow{a} y \xrightarrow{b} z$, and the residual footprints of a and b are disjoint. Then b is legal at x , a is legal after b , both orders reach the same state, and the two footprints are unchanged by swapping the order.
- (L2) **Terminal locality.** If $x \xrightarrow{a} y$ and a terminal reduction r is legal at one endpoint with footprint disjoint from $\Phi_x(a)$, then its corresponding residual is legal at the other endpoint with the same footprint.

(L3) **Reversibility.** Every neutral action has a local inverse whose footprint is the same bounded set of sites.

The backward half of (L1) is essential. It says that a move observed after an unrelated action was already legal before that action. This is what permits a birth action to commute left.

Remark 3.4. For a concrete implementation, one can use a conservative footprint larger than the minimal topological support. This can only classify more actions as interacting. The search becomes less sharply pruned, but all commutation claims remain safe.

3.3 Incidence bounds

We separate the number of moves available globally from the number meeting a small active region.

Definition 3.5 (Bounded incidence). A rule family has constants $A, B > 0$ when, for every state x ,

$$\begin{aligned} |\{a : a \text{ neutral and legal at } x\}| &\leq AN, \\ |\{a : a \text{ neutral and legal at } x, \Phi_x(a) \cap S \neq \emptyset\}| &\leq B|S| \end{aligned}$$

for every $S \subseteq V$.

The first bound permits $O(N)$ possible locations for a new independent branch. The second says that continuation from a region of $O(k)$ sites has only $O(k)$ choices. In a finite local rule system on a bounded-degree combinatorial structure, such constants are natural.

3.4 Unlocking traces

Let

$$x_0 \xrightarrow{a_1} x_1 \xrightarrow{a_2} \cdots \xrightarrow{a_k} x_k$$

be a neutral trace.

Definition 3.6 (First-unlocking trace). The trace is first-unlocking when no terminal reduction is legal at $x_0, \dots, x_{k-1}$ and at least one terminal reduction r is legal at x_k . Its length is k . It is *shortest* when no first-unlocking trace from x_0 has smaller length.

A search can stop at the first terminal reduction. Therefore any trace that exposes a reduction earlier than expected is automatically replaced by its shorter prefix.

Definition 3.7 (Birth and birth number). Write $\Phi_i = \Phi_{x_{i-1}}(a_i)$. Action a_i is a *birth* when

$$\Phi_i \cap \bigcup_{j < i} \Phi_j = \emptyset.$$

The birth number $\beta(\tau)$ is the number of birth actions in τ .

The first action is always a birth. A birth is not necessarily topologically remote forever: later actions may connect its footprint to other branches.

3.5 The action-support graph

Fix a terminal reduction r legal at x_k . The action-support graph $H(\tau, r)$ has vertices $1, \dots, k, \star$. Vertex i represents a_i and $\star$ represents r . Put an edge between two action vertices when their executed footprints intersect, and an edge $i\star$ when Φ_i intersects the terminal footprint $\Psi_{x_k}(r)$ after the canonical residual identification supplied by strong locality.

In the fixed-site situations used here, the residual identification is literal: the same dart labels are compared throughout. The graph records causal noncommutation, not chronological adjacency.

Example 3.8 (Two branches before a join). Let actions A and B have disjoint footprints. Let C overlap both, and let the terminal reduction overlap C . The chronological word ABC has two support components after its second letter, but the full action-support graph is connected. This is the smallest obstruction to the naive claim that every shortest trace can be given connected prefixes.

4 Causal structure and the birth-front normal form

4.1 Shortest traces have connected dependency graphs

Theorem 4.1 (Causal connectedness). *Assume strong disjoint commutation. If τ is a shortest first-unlocking trace and r is a terminal reduction exposed at its end, then $H(\tau, r)$ is connected.*

Proof. Suppose $H(\tau, r)$ is disconnected. Let C be a connected component not containing the terminal vertex $\star$. Every action in C has footprint disjoint from every action outside C and from the terminal reduction.

Use the commutation diamond repeatedly to move every action of C to the right, preserving the relative order inside C and the relative order of all actions outside C . Each adjacent swap is legal, preserves the reached state, and preserves the relevant footprints. We obtain a trace-equivalent word

$$\tau' \sigma_C,$$

where σ_C consists exactly of the actions in C .

The terminal reduction is legal after $\tau' \sigma_C$. Since its footprint is disjoint from every footprint in σ_C , terminal locality moves it left across the entire block. Hence it is already legal after τ' . Deleting σ_C gives a strictly shorter trace that exposes a terminal reduction; if one was exposed at an earlier prefix of τ' , truncate at its first such prefix. In either case we obtain a strictly shorter first-unlocking trace from x_0 , contradicting the minimality of τ . $\square$

Corollary 4.2. *Every birth in a shortest trace has a path, through overlapping later action footprints, to the terminal reduction.*

This corollary is qualitative. It rules out irrelevant disconnected work, but it does not bound the number of births. A connected dependency tree can have many leaves.

4.2 Why connected prefixes are false

The following proposition records the proof-audit failure that led to the birth parameter.

Proposition 4.3 (Branching obstruction). *There exists a strongly local reversible rewrite system with a shortest first-unlocking trace of length three such that every first-unlocking trace of length three has birth number two. In particular, no commuting-equivalent first-unlocking trace has connected chronological prefixes.*

Proof. Use three Boolean sites 0, 1, 2, initially all zero. Action A toggles site 0 and has footprint $\{0\}$. Action B toggles site 2 and has footprint $\{2\}$. Action C is legal exactly when sites 0 and 2 are one; it toggles site 1 and has footprint $\{0, 1, 2\}$. The terminal reduction is legal exactly when site 1 is one.

The moves are involutions, and disjoint supports commute. Neither A nor B alone enables C , so at least two moves are required before C . The words ABC and BAC unlock the reduction in three moves. Their first two footprints are disjoint, so each has two births. No trace with one birth can set both sites 0 and 2 before applying C . $\square$

The fixture is implemented as `branching_example()` in the prototype. It is not asserted to be a knot diagram; its purpose is to refute an invalid general lemma before that lemma is imported into knot topology.

4.3 Births commute to the front

Lemma 4.4 (Births are pairwise disjoint). *The footprints of distinct births in one trace are disjoint.*

Proof. A later birth is disjoint from the union of every earlier footprint, including the footprint of any earlier birth. $\square$

Lemma 4.5 (Left commutation of a birth). *A birth action can be commuted left across every action preceding it.*

Proof. By definition, its footprint is disjoint from each preceding footprint. Apply the backward-and-forward commutation diamond one adjacent predecessor at a time. Footprints remain unchanged under each swap, so the next swap remains valid. $\square$

Theorem 4.6 (Birth-front normal form). *Let τ be a first-unlocking trace of length k and birth number b in a strongly local reversible system. There is a first-unlocking trace $\hat{\tau}$ of length at most k and birth number at most b such that:*

1. *all births of $\hat{\tau}$ occur before every nonbirth action;*
2. *after the birth block, every action footprint intersects the union of all earlier footprints.*

Proof. Identify the births in the original trace. By Lemma 4.5, move each birth left across all predecessors. By Lemma 4.4, births commute with one another, so choose any fixed canonical order for them. Preserve the relative order of all nonbirth actions.

Consider an original nonbirth action a_i . It overlapped at least one earlier action. If that earlier action was a birth, it has moved farther left. If it was a nonbirth, their relative order was preserved. Thus a_i still meets the accumulated earlier footprint in the reordered word. Strong commutation shows that the reordered word is legal and reaches the same final state.

It is possible that the terminal reduction becomes legal at an earlier prefix of the reordered word. In that case truncate at the first such prefix. The length and number of births can only decrease. The truncated word is first-unlocking and has the two stated properties. $\square$

Remark 4.7 (Normal form versus a chronological invariant). The birth number is defined on a trace, not just its final state. The theorem shows that it has algorithmic meaning: it is an upper bound on the number of global choices needed before all remaining choices are local. One may minimize b over shortest traces to obtain an input parameter, but the search does not need to know that minimum in advance; it can deepen in b .

4.4 Accumulated support strictly improves last-touch search

The current ProveIt recursion follows the neighborhood of the immediately previous RIII move. The next proposition shows that even the one-birth normal form is strictly more general in local rewrite systems.

Proposition 4.8 (Strict separation). *There is a strongly local reversible system with a one-birth first-unlocking trace of length three, but no first-unlocking trace of length at most three in which consecutive action footprints always intersect.*

Proof. Use sites $0, 1, 2, 3$, initially zero. Action A toggles sites 0 and 1 and has footprint $\{0, 1\}$. Action B requires site $0 = 1$, toggles site 2 , and has footprint $\{0, 2\}$. Action C requires site $1 = 1$, toggles site 3 , and has footprint $\{1, 3\}$. The terminal reduction requires sites $2 = 3 = 1$.

After A , both B and C are legal. They commute because their footprints are disjoint. Either ABC or ACB unlocks the terminal reduction. There is one birth, namely A , and each leaf meets the accumulated footprint of A . However, B and C do not meet one another, so neither length-three ordering has pairwise-intersecting consecutive footprints. No shorter word can set both sites 2 and 3 . $\square$

The fixture is `accumulated_strict_example()` in the prototype. For knot diagrams, this proposition motivates the accumulated-footprint search; it does not by itself exhibit a concrete RIII diagram with the same separation.

5 Parameterized enumeration and asymptotic regimes

5.1 The birth-front search

The normal form leads to a direct iterative-deepening algorithm.

Algorithm 5.1 (Birth-front unlocking search). Given a state x , a depth bound k , and a birth bound b :

1. For $\ell = 1, \dots, k$, search traces of length at most ℓ .
2. Enumerate a canonical list of at most b pairwise footprint-disjoint births. Each birth is chosen among all currently legal neutral actions.
3. After the birth block, permit only a legal action whose footprint meets the union of all previously chosen footprints.

4. After every neutral action, test for a terminal reduction. Return the shortest successful trace.
5. Undo failed branches exactly. Exhaustion returns *unknown*, never a negative verdict.

A canonical order on births removes permutations of pairwise commuting actions. The complexity theorem below does not need that saving, so its bound is robust to a simpler implementation.

5.2 Counting search nodes

Theorem 5.2 (Parameterized enumeration). *Assume footprint size at most f , global incidence at most AN , local incidence at most $B|S|$, and polynomial-time state operations. The birth-front algorithm finds a first-unlocking trace whenever one of length at most k and birth number at most b exists, and exhausts all birth-front normal-form candidates in time*

$$N^{b+O(1)}(Ck)^k, \quad C = Bf \max(1, A),$$

and polynomial working memory. The constant implicit in $O(1)$ is independent of b and k .

Proof. By Theorem 4.6, it is enough to enumerate traces with all births first. Suppose a normal-form trace has $r \leq b$ births and total length $\ell \leq k$.

At each birth position there are at most AN legal neutral actions, so the birth block has at most $(AN)^r$ possibilities. After j actions have been chosen, the accumulated footprint contains at most fj sites. Bounded local incidence therefore gives at most Bfj possible connected extensions. Consequently, the number of length- ℓ search words with r births is at most

$$(AN)^r \prod_{j=r}^{\ell-1} Bfj \leq (AN)^b (Bfk)^k.$$

Summing over $r \leq b$ and $\ell \leq k$ contributes at most a polynomial or $2^{O(k)}$ factor, which is absorbed by replacing C with a larger absolute rule-set constant.

Each node performs polynomial-time legality, application, terminal, and undo operations. A depth-first traversal stores one state representation, at most k undo records, the accumulated footprint of size at most fk , and polynomial bookkeeping. Hence the working memory is polynomial in N and k . $\square$

Remark 5.3 (Why the exponent b appears). Every birth can be located anywhere in the diagram, so a direct deterministic search pays one factor of N per independent branch. Removing this exponent would require extra structure, such as color-coding, representative families, a reverse terminal-rooted search, or a geometric theorem constraining where births can occur.

5.3 Quasi-polynomial and polynomial corollaries

Write

$$T(N; b, k) = N^{b+O(1)}(Ck)^k.$$

Taking logarithms gives

$$\log T = (b + O(1)) \log N + k \log(Ck).$$

Corollary 5.4 (Logarithmic parameters). *If $b = O(\log N)$ and $k = O(\log N)$, then*

$$T(N; b, k) = \exp(O((\log N)^2)) = N^{O(\log N)}.$$

Thus the search is quasi-polynomial.

Corollary 5.5 (Constant branching). *If $b = O(1)$ and $k = O(\log N)$, then*

$$T(N; b, k) = \exp(O(\log N \log \log N)) = N^{O(\log \log N)}.$$

Corollary 5.6 (Sublogarithmic depth). *If $b = O(1)$ and $k = O(\log N / \log \log N)$, then $T(N; b, k) = N^{O(1)}$.*

Proof. For the last statement, $k \log(Ck) = O(\log N)$, while the birth factor is polynomial because b is constant. $\square$

5.4 Search budgets

A practical implementation will cap the number of trial moves rather than run the worst-case tree. The theorem and the budget serve different purposes.

- The theorem says what is complete when the specified (b, k) tree is exhausted.
- A trial budget may stop earlier to protect the exact fallback.
- Budget exhaustion has no mathematical meaning about knottedness or about the nonexistence of a trace.
- Successful traces remain exact because they are independently replayed.

This is compatible with the current ProveIt design, which already treats RIII search as an optional pre-scan accelerator.

6 Specialization to Reidemeister-III unlocking

We now verify the abstract hypotheses for the mutable dart representation used by the reviewed ProveIt code. The purpose is not to reprove Reidemeister’s theorem; it is to account for the local search tree.

6.1 Fixed darts and the face permutation

An N -crossing diagram has darts

$$V = \{0, 1, \dots, 4N - 1\}.$$

Darts $4i, \dots, 4i + 3$ belong to crossing i . Let α be the fixed-point free involution pairing the two darts of each diagram edge. Let ρ rotate a dart to the next slot at its crossing. A face walk is generated by

$$\varphi = \rho \circ \alpha.$$

The implementation writes this map as `nxt`. A triangular face is a three-cycle of φ through three distinct crossings. The over/under slot parities determine whether that triangle is an admissible RIII configuration.

The RIII update keeps crossing identities and slots fixed and modifies only the edge involution. This is particularly convenient for the causal analysis: site names do not move even though adjacency does.

6.2 A conservative RIII footprint

Let T be a legal RIII triangle with crossing set $C(T)$ of size three. Define

$$D(T) = \{4c + j : c \in C(T), 0 \leq j < 4\}$$

and the conservative footprint

$$\Phi(T) = D(T) \cup \alpha(D(T)).$$

Thus $|\Phi(T)| \leq 24$.

Lemma 6.1 (Footprint containment). *The reviewed `apply_r3` routine changes α only on darts in $\Phi(T)$. Its inverse has the same footprint set.*

Proof. The routine names the six outer slots of the three participating strands and the corresponding inner slots. Every new link has one endpoint at a dart of $D(T)$; its other endpoint is either another dart of $D(T)$ or the previous α -partner of a dart in $D(T)$. Hence every written array entry lies in $D(T) \cup \alpha(D(T))$. Moreover, each outside dart in $\alpha(D(T)) \setminus D(T)$ remains paired to some dart of $D(T)$ after the move, while no new outside dart is introduced. Therefore the post-move set $D(T) \cup \alpha'(D(T))$ equals the pre-move footprint. RIII is an involution, so the reverse action has exactly this same footprint. $\square$

For RI/RII terminal tests, use the analogous conservative footprint consisting of all darts at the one or two participating crossings together with their edge partners. Its size is bounded by 16.

6.3 Disjoint footprints commute

Proposition 6.2 (Strong locality for the dart rules). *RIII actions and RI/RII terminal tests in the fixed-dart implementation satisfy strong disjoint commutation when the conservative footprints above are used.*

Proof. An RIII action writes only entries of α in its footprint by Lemma 6.1. Suppose a second RIII action is observed after the first and their residual footprints are disjoint. The twelve darts owned by the second crossing triple lie outside the first footprint. Since the first move changes α only inside its own footprint, the partners of those twelve darts are also unchanged; hence the second conservative footprint, its three-cycle face test, and its slot-parity test were already identical before the first move. The same argument in the other direction proves the two-sided legality diamond. The two updates write disjoint array entries, so they commute entry by entry and their footprints remain fixed under the swap.

The same argument applies to RI/RII legality: monogon and bigon face walks read only the darts at their participating crossings and their partners. A disjoint RIII update neither creates nor

destroys such a local face. Equivalently, any RI/RII move newly exposed by an RIII update has a conservative footprint intersecting that RIII footprint. Finally, RIII inversion is its own local inverse with the same footprint. $\square$

Remark 6.3. Two RIII moves with disjoint *crossing triples* need not have disjoint conservative footprints, because their local disks can touch the same outside edge. This is why the theorem is formulated with dart footprints rather than only crossing labels.

6.4 Global and local incidence

For a connected spherical 4-regular projection graph with N vertices, $E = 2N$ and Euler's formula gives $F = N + 2$ faces. A legal RIII action is supported by a triangular face, so the total number of legal RIII locations is at most $N + 2$. This gives the global incidence bound with an absolute A .

For the local bound, fix one dart d . If $d \in \Phi(T)$, then either a crossing of T owns d , or a crossing of T owns $\alpha(d)$. Each crossing has four face corners, so it is incident to at most four triangular faces. Consequently at most eight triangular faces have footprints containing a given dart. By a union bound,

$$|\{T : \Phi(T) \cap S \neq \emptyset\}| \leq 8|S|.$$

Thus one may take $f = 24$ and $B = 8$, giving $Bf = 192$. Whenever an RIII location exists, $N \geq 3$ and $N + 2 \leq 2N$, so a canonical enumeration with one representative per triangular face may take $A = 2$ and hence $C = 384$ in Theorem 5.2. An implementation that emits orientations or duplicate representatives is covered by some larger absolute C ; no numerical optimality is claimed, and the asymptotic regimes are unchanged.

Corollary 6.4 (RIII search bound). *On an N -crossing fixed-dart state, a first-unlocking RIII trace of length at most k and birth number at most b , when one exists, can be found by exhaustive birth-front search in*

$$N^{b+O(1)}(Ck)^k$$

time and polynomial memory for an absolute constant C .

6.5 Why accumulated footprints are the right implementation object

The existing recursive search computes a neighborhood from the triangle just applied and recursively scans that neighborhood. This is a last-touch chain. For birth number one, the complete normal-form search must instead retain the union of all previous RIII footprints. A candidate is local when its current footprint intersects that union. The active set contains at most $24k$ darts, so scanning crossings owning active darts and their current partners generates only $O(k)$ local triangle candidates, up to duplicates.

For $b > 1$, all births are chosen first. They are required to have pairwise disjoint footprints and are ordered canonically. After the first connected move, no further birth is permitted. This exactly implements Theorem 4.6.

6.6 Interaction with the face shortcut

The current implementation's `r3_can_help` test is used only for the last allowed RIII move. It checks a necessary local condition for that move to create an RI/RII face. Retaining that placement

is important. Applying the shortcut to every intermediate move would be unsound, because an intermediate RIII need not immediately expose a reduction; it may only prepare a later interaction.

6.7 State restoration

Every unsuccessful branch must restore the exact involution. The reviewed code undoes an RIII move by locating the inverse triangle and applying the same involution routine. The adapter preserves this contract. A production integration should additionally test every budget-exhaustion boundary, because an early return from a recursive branch is a common place for mutable-state leaks.

7 The restricted recognizer and integration design

7.1 A monotone simplification class

Fix functions $b(N)$ and $k(N)$, a deterministic RI/RII exhaustion order, and a deterministic ordering of the birth-front search tree (for example, iterative depth followed by lexicographic triangle keys). Let $\mathbf{BF}_{b,k}$ denote the resulting deterministic preprocessor. Define $\mathcal{U}_{b,k}^{\mathbf{BF}}$ to be the set of unknot diagrams on which this specific process reaches the crossing-free circle:

1. exhaust RI/RII in the fixed order;
2. if crossings remain, run the complete birth-front search with bounds $b(N), k(N)$ and select its first trace in the fixed ordering;
3. apply that trace, return to step 1, and return **Unknown** if no trace is found.

The policy qualification is necessary. Two different successful unlock traces can lead to different reduced diagrams, and the existence of one globally good sequence does not by itself justify a greedy choice without backtracking. The definition permits the diagram size to fall during the process; using the original N as the upper-bound parameter only weakens the estimate.

Theorem 7.1 (Restricted-class unknot recognition). *There is a deterministic, sound algorithm that returns **Unknot** on every diagram in $\mathcal{U}_{b,k}^{\mathbf{BF}}$ and otherwise returns either **Unknot** or **Unknown**. Its running time is*

$$N^{b(N)+O(1)}(Ck(N))^{k(N)}$$

and its working memory is polynomial, apart from any subsequent fallback.

Proof. At each stalled state, run the complete birth-front search with the specified bounds and fixed ordering. If it succeeds, select the first trace in that ordering, replay it, expose RI/RII, and exhaust reductions in the fixed order. Reidemeister moves preserve the represented knot type. Therefore reaching the crossing-free circle is a sound unknot certificate.

Each successful episode is followed by an RI or RII move and hence removes at least one crossing. There are at most N episodes. Multiplying the single-episode bound from Corollary 6.4 by N is absorbed in the $N^{O(1)}$ factor. If a search fails, return **Unknown** or invoke an exact fallback; no negative conclusion is drawn. $\square$

Corollary 7.2. *The policy class $\mathcal{U}_{b,k}^{\text{BF}}$ is recognized in quasi-polynomial time when $b(N), k(N) = O(\log N)$, in $N^{O(\log \log N)}$ time when $b(N) = O(1)$ and $k(N) = O(\log N)$, and in polynomial time when $b(N) = O(1)$ and $k(N) = O(\log N / \log \log N)$.*

7.2 Certificate size and replay

A successful preprocessor emits the same kind of trace already used by ProveIt: for each RIII move, the three stable input crossing indices, followed by RI/RII records. Across at most N reduction episodes, a uniform depth bound k gives $O(Nk)$ RIII records. Each crossing index uses $O(\log N)$ bits, so the certificate size is $O(Nk \log N)$ bits. The existing independent replay routine checks legality move by move and rebuilds a validated spherical one-component diagram.

This proof-carrying behavior is more important than the heuristic that found the trace. Search policies, budgets, and branch ordering may change without changing the trusted replay kernel.

7.3 Opt-in API

The supplied review patch adds two optional arguments to `simplify`:

Argument	Suggested values	Meaning
<code>r3_search</code>	"last", "clustered"	legacy or birth-front search
<code>r3_births</code>	positive integer	maximum number of births

The default remains "last"; therefore applying the patch alone does not change existing behavior. In clustered mode, `r3_births=1` gives the accumulated-support search. Larger values admit independent birth branches. The existing `r3_depth`, `r3_budget`, and face-shortcut controls are retained.

7.4 Pseudocode

A simplified version of one depth-limited search is shown below.

Listing 1: Birth-front RIII search, schematic

```

1 def search(state, depth_left, births_left, birth_phase,
2   active_footprint, last_birth_key, path):
3   if budget_exhausted() or depth_left == 0:
4       return failure
5
6   if birth_phase and births_left > 0:
7       for T in all_legal_triangles(state):
8           F = footprint(state, T)
9           if F intersects active_footprint:
10              continue
11           if key(T) <= last_birth_key:
12              continue
13           apply(T)
14           if RI_or_RII_now_legal(F):
15              return success

```

```

16         if search(state, depth_left - 1, births_left - 1,
17                   True, active_footprint union F, key(T), path + [T]):
18             return success
19         undo(T)
20
21     for T in legal_triangles_meeting(state, active_footprint):
22         F = footprint(state, T)
23         apply(T)
24         if RI_or_RII_now_legal(F):
25             return success
26         if search(state, depth_left - 1, births_left,
27                 False, active_footprint union F,
28                 last_birth_key, path + [T]):
29             return success
30         undo(T)
31
32     return failure

```

The production helper contains duplicate suppression, inverse-move avoidance, trial accounting, and the last-move face shortcut.

7.5 Required production tests

Before clustered mode can be enabled by default, the following checks are mandatory.

1. **Trace replay.** Replay every successful trace with the existing independent checker.
2. **Exact restoration.** Hash the entire α involution before and after every failed search and every budget cutoff.
3. **Invariant preservation.** Compare Alexander data, writhe, and Khovanov ranks on randomized successful RIII traces, as the existing synthesis did for the legacy helper.
4. **Verdict equality.** Compare the full recognizer with clustered mode disabled and enabled on maintained and random corpora.
5. **Adversarial branching.** Add fixtures requiring two disjoint birth footprints before a connecting RIII move, if a genuine knot-diagram example can be found.
6. **Performance separation.** Measure successes and failures separately. Deep failed searches should remain budgeted and should run only immediately before the expensive exact scan.

7.6 Why the exact fallback remains essential

Even a complete (b, k) search is incomplete for unrestricted unknot diagrams. If it fails, the diagram may still be an unknot that needs larger parameters, crossing-increasing moves, a different diagram language, or a non-Reidemeister certificate. The ProveIt pipeline should therefore keep its exact Khovanov recognizer and every sound obstruction. Clustered RIII search is a front-end optimization with a theorem, not a replacement decision procedure.

8 Executable validation and negative controls

The mathematical proof does not depend on experiments. The experiments serve three narrower purposes: catch implementation mistakes, demonstrate the two strict separations, and make the scope of evidence explicit.

8.1 Reference implementation

The file `prototype/causal_search.py` implements a finite class of Boolean local rewrite systems. A neutral action has:

- a stable support;
- guards that read only support sites;
- toggles on support sites not read by the guards.

Applying a legal action twice restores the state. Actions with disjoint supports commute and preserve one another’s legality. A terminal reduction is a local bit-pattern predicate.

Three searches are included.

1. unrestricted breadth-first search, used as a finite oracle;
2. the birth-front search proved complete under the parameter bound;
3. a last-touch search modeling the current immediate-predecessor locality restriction.

8.2 Hand-checked strict examples

The two examples from Propositions 4.3 and 4.8 are executable. A third fixture checks a subtler implementation point: accumulated support is search state even when the physical rewrite state repeats.

Fixture	Witness depth	Minimum births	Distinguished failure
Two independent births, then join	3	2	one-birth search fails
One birth, two commuting leaves	3	1	last-touch search fails
State repeat extends support	4	1	state-only pruning fails

These are negative controls. They ensure the implementation is not silently reinstating either rejected claim.

8.3 Generated finite systems

The deterministic validation script generated 5,000 systems with six Boolean sites, nine neutral actions, three terminal reductions, and search depth at most six. The random seed is the system index. The outcome was:

Category	Count
Initially reducible	3,209
No witness within depth six	131
Witness found by unrestricted BFS	1,660
Front-loading failures	0
Birth-front completeness discrepancies	0
Disconnected shortest dependency graphs	0

Among the 1,660 witnessed systems, the unrestricted oracle returned depths 1, 2, 3, 4, 5 with counts 1381, 223, 47, 8, 1. The birth-front search needed birth bound one in 1,603 cases and birth bound two in 57 cases. The oracle’s first returned shortest trace had birth counts one, two, and three in 1,548, 110, and two cases respectively; a different shortest trace sometimes used fewer births.

The unrestricted BFS explored 3,863 dequeued states across these runs; the iterative birth-front comparisons explored 12,457 recursive nodes. This is not a speed benchmark: the state space has only 64 states, BFS deduplicates globally, and the parameterized search deliberately mirrors a local depth-first implementation. The relevant result is equality of witnessed behavior, not a runtime win on toy Boolean systems.

8.4 Exhaustive three-site catalog

A second script exhausts a fully specified finite catalog rather than sampling. There are 27 one-bit involution templates on three sites: choose the toggled site, and for each of the other two sites choose guard value zero, guard value one, or absence. The script checks every three-move subset, every one of the 26 nonempty local terminal predicates, and all eight initial states, for a total of 608,400 instances through depth four.

Catalog outcome	Count
Initially reducible	163,800
No witness within depth four	272,632
Witnessed	171,968
Front-loading failures	0
Birth-front completeness discrepancies	0
Disconnected shortest dependency graphs	0

This catalog is still only a finite software check, but it removes sampling uncertainty inside the stated model class. The exact parameters and counts are stored in `results/exhaustive_catalog.json`.

8.5 Unit tests

Five generic-search and two adapter-contract checks cover:

1. the two-birth branching fixture;
2. strict improvement of accumulated support over last-touch locality;
3. preservation or shortening under front-loading;

4. the support-history state-repeat regression;
5. 200 additional generated-system oracle comparisons;
6. exact adapter restoration after failure; and
7. footprint-stable inverse replay after adapter success.

All seven checks pass under the Python interpreter in the artifact environment.

8.6 What was not tested

No full ProveIt checkout was available to the container runtime. Consequently:

- the 431 maintained repository tests were not rerun;
- the adapter was exercised only against a twelve-dart contract gadget, not actual validated PD diagrams;
- no timing claim is made for clustered RIII search;
- no knot fixture is claimed to separate last-touch and accumulated support;
- no corpus claim is extrapolated from the Boolean model.

The integration snippet was syntax-checked and its success/failure mutation contract was tested against a twelve-dart implementation of the reviewed RIII formulas. Its baseline source and blob hashes are pinned so that repository-side review can detect drift.

8.7 Interpretation

The validation supports four modest conclusions.

1. The generic implementation agrees with the theorem on a substantial finite sample.
2. The false connected-prefix conjecture is concretely guarded against.
3. The accumulated-support algorithm is genuinely different from the legacy last-touch algorithm.
4. Production correctness still depends on repository-native replay, restoration, invariant, and regression tests.

9 Consequences, comparisons, and limitations

9.1 A new structural target

Earlier conditional routes in the project parameterize exact computation by frontier width, twist-run count, extremal Khovanov windows, transfer rank, or hierarchy data. The present result adds a parameter on the diagram-rewriting side:

(birth number, unlock depth).

It is independent of homological width. A diagram may have a difficult exact scan but a small causal RIII certificate, or conversely a simple exact invariant calculation but no short monotone simplification.

This separation suggests an adaptive policy: estimate local RIII branch growth and exact-scan frontier growth, then spend a bounded amount on whichever has the more favorable certified parameter. Such a policy must preserve the existing rule that abandoned work cannot affect verdicts.

9.2 Why the result is stronger than a depth-only bound

A naive global depth- k RIII search has $O(N)$ choices at each level and hence $N^{O(k)}$ behavior. At $k = O(\log N)$ this is already $N^{O(\log N)}$, quasi-polynomial, but with a large exponent and no recognition of local causal structure. More importantly, the current implementation does not perform that complete global search because it would be too expensive.

The causal theorem decomposes the exponent. Only births pay a factor of N ; all connected continuations pay $O(k)$. When b is constant and $k = O(\log N)$, the exponent improves from $O(\log N)$ to $O(\log \log N)$. This is the main asymptotic gain.

9.3 Why polynomial Reidemeister length is not enough

Lackenby's polynomial move bound permits crossing increases and controls a much broader class of sequences. A polynomial total length N^{11} inserted into our enumeration formula would be useless. The present theorem needs two finer facts:

1. the neutral run between decreases is short;
2. only a few support-disjoint branches are born during that run.

Proving these facts for all unknots would be a major additional theorem and may be false for the strictly crossing-nonincreasing move language.

9.4 Potential relation to planar separators

The active RIII footprint contains $O(k)$ darts and induces a small subgraph of the projection. If this active region admits balanced separators that are stable under local moves, one could attempt a divide-and-conquer search in the style of frontier algorithms. Such a theorem might replace the N^b birth factor by a smaller separator-dependent term. At present, however, a birth can occur anywhere, and no safe separator theorem for the dynamic RIII causal graph is proved here.

9.5 Potential relation to hierarchy certificates

A short RIII causal trace is a two-dimensional diagram certificate; an essential hierarchy is a three-dimensional complement certificate. There may be a transfer principle: a uniformly local hierarchy simplification could constrain where diagrammatic births occur, while a small causal footprint might identify a bounded part of a handle structure that needs modification. The 2026 hierarchy algorithm makes this a timely direction, but no such bridge is claimed.

9.6 Lower-bound questions

The birth parameter also supports negative results. To prove that this route cannot solve the unrestricted problem, one could construct unknot diagrams for which every crossing-nonincreasing first-unlocking trace has either large depth or large birth number. The connectedness theorem says a shortest trace cannot contain causally irrelevant branches, so a lower bound on births would be a meaningful topological obstruction rather than a search artifact.

9.7 Soundness boundary

The preprocessor has one-sided semantics:

successful replayable trace $\Rightarrow$ **Unknot**, failure or timeout $\Rightarrow$ **Unknown**.

It cannot certify knottedness. Knottedness remains the responsibility of sound invariants or exact complement/homology methods. This boundary should be preserved in every CLI and serialized-certificate interface.

9.8 Assessment of the research contribution

Within its scope, the report delivers four concrete advances.

1. It replaces an empirical locality heuristic by an explicit two-parameter completeness theorem.
2. It proves a restricted quasi-polynomial and polynomial classification.
3. It identifies and repairs a subtle but natural false proof strategy.
4. It supplies code and a low-risk opt-in integration path.

The unresolved step is geometric: establish useful upper bounds on births and depth for broad, natural families of unknot diagrams, or design a different certificate when those parameters are large.

10 Further research questions and proposed program

The following questions are ordered roughly from immediate engineering tasks to longer-term topology. They are phrased so that a negative answer would also be informative.

10.1 Immediate repository work

- Q1. Find a genuine accumulated-support separator.** Does there exist a PD-code unknot for which the reviewed last-touch search fails at a given depth but the one-birth accumulated-footprint search succeeds at that depth? An exhaustive search over small diagrams would turn Proposition 4.8 from an abstract warning into a knot-specific regression fixture.
- Q2. Find a genuine two-birth RIII unlock.** Is there an unknot diagram whose shortest first-unlocking RIII trace has birth number at least two with the conservative dart footprint? If so, minimize crossings and depth and add the result as an adversarial test.

- Q3. Measure the empirical parameter distribution.** On the existing random closures, scrambled unknots, hard survivors, and named examples, record the minimum successful pair (b, k) under a fixed trial cap. Separate successful and failed search costs, and publish raw traces rather than only aggregate counts.
- Q4. Incremental candidate maintenance.** Can legal RIII triangles meeting the active footprint be updated in amortized constant time per move, rather than rediscovered from nearby darts? The dart involution changes only a bounded footprint, so a local invalidation table should be possible.
- Q5. Canonical commutation reduction.** Beyond sorting births, can independent connected moves be quotiented by a trace-monoid normal form? A safe partial-order reduction could remove many permutations without changing completeness.

10.2 Parameterized topology

- Q6. Bounded births for natural diagram classes.** Are reduced alternating unknot diagrams (necessarily trivial after reduction), positive braid-derived unknots, arborescent unknots, or diagrams of bounded projection-treewidth guaranteed to have bounded birth number during monotone simplification?
- Q7. Logarithmic depth from bounded width.** Does bounded treewidth, branchwidth, carving width, or pathwidth of the projection graph imply $k = O(\log N)$ after an appropriate choice of crossing-decreasing reduction order? A balanced-separator proof would directly combine with the present theorem.
- Q8. Birth number versus dependency-tree width.** The support-overlap graph of a shortest trace is connected. Relate the minimum birth number to standard directed or undirected parameters of this graph: number of sources under chronological orientation, pathwidth, vertex separation, or search number. Which parameter admits the best deterministic enumeration?
- Q9. Lower bounds.** Construct families of unknot diagrams requiring $\Omega(\log N)$, $\Omega(N^\epsilon)$, or $\Omega(N)$ births or RIII depth under crossing-nonincreasing RI/RII/RIII simplification. Such a family would sharply delimit this route.
- Q10. Reduction-order optimization.** The current simplifier takes available RI/RII moves greedily. Can a different choice of decreasing move reduce the later birth/depth parameters? Determine the complexity of finding an optimal reduction order and whether local confluence or matroid-like exchange principles apply.

10.3 Algorithmic improvements to the birth exponent

- Q11. Color-coding independent births.** Can randomized or derandomized color-coding find b disjoint causal birth footprints in $2^{O(b)} \text{poly}(N)$ rather than N^b time, while still accounting for their later joins?
- Q12. Terminal-rooted reverse search.** RIII is an involution. Can one enumerate causal predecessors backward from potential RI/RII footprints, meeting a forward search in the middle? The obstacle is that the final diagram is unknown; a bounded local-state interface might overcome it.

- Q13. Representative families.** When several birth sets induce the same boundary behavior on an active footprint, can all but a small representative family be discarded? This would parallel rank-based or connectivity-based dynamic programming.
- Q14. Planar localization.** Prove that a shortest birth footprint can be chosen inside a bounded-radius neighborhood of the eventual connecting action, or find a counterexample. A positive theorem would replace global N choices by a function of k .
- Q15. Learning only as a scheduler.** Can a learned model predict which birth candidates to try first while the complete bounded search remains available underneath? The learned layer must affect order only, never pruning or verdicts.

10.4 Combination with exact invariants

- Q16. Khovanov-aware RIII ordering.** Can inexpensive frontier estimates predict which legal RIII move will reduce later Khovanov scan width, not merely crossing count? The synthesis already observes that fewer crossings can increase scan work.
- Q17. Joint parameter theorem.** Suppose RIII birth/depth are moderately large but the resulting diagram has a small exact-scan frontier. Derive a combined bound that branches on RIII only until a certified frontier threshold is reached, then switches to exact contraction.
- Q18. Twist-run compression after causal simplification.** Does a small RIII causal footprint preserve or reveal a low signed-braid run count? If so, the twist-compression theorem and the present theorem could be composed rather than run as unrelated alternatives.
- Q19. Hierarchy transfer.** Relate a bounded RIII causal region to a bounded modification of a handle hierarchy or boundary pattern. This could provide a three-dimensional certificate that replaces a large remaining diagram search.

10.5 Formal verification

- Q20. Lean formalization of the abstract theorem.** Formalize strong disjoint commutation, causal connectedness, birth front-loading, and the counting bound. The Boolean prototype supplies a finite executable semantics, but the proof should be independent of it.
- Q21. Verified dart footprint.** Prove that the concrete `apply_r3` function writes only the claimed 24-dart footprint and is an involution on every validated diagram.
- Q22. Proof-carrying traces.** Specify a stable JSON schema for RIII/RI/RII traces, source hashes, budgets, and replay results. Make the replay kernel the only trusted component for positive simplification verdicts.

10.6 A focused next milestone

The most informative next experiment is not to raise depth globally. It is to implement opt-in $b = 1$ accumulated support, preserve the current budget, and run paired tests on the exact 117-depth corpus recorded by the synthesis. Three outcomes are all useful:

- new successes establish that last-touch locality was a real completeness bottleneck;

- no new successes but low overhead suggests the stronger search can safely replace the heuristic;
- high failed-search overhead identifies the need for partial-order reduction or a better scheduler before increasing b .

A Reproducibility, audit trail, and exact commands

A.1 Reviewed source pins

Item	Pin
ProveIt repository	VladimirReshetnikov/ProveIt
Reviewed commit	23893144e3aafd074a23cfb633222ab7e03e6d2e
RIII source path	Topology/UnknotRecognition/fast/fastunknot/simplify.py
RIII source blob	2fcaabb51eaa45cf84bdee32cd1b4e18acf2c478
OpenAI math README blob	8ac0cb7dbf1f38ad3b7480310d4e1d9482f0cfe8
Date of review	October 7, 2026, America/Los_Angeles

A.2 Bundle commands

From the artifact root:

```

1 python -m unittest discover -s tests -v
2 python experiments/exhaustive_validation.py \
3   --systems 5000 \
4   --sites 6 \
5   --moves 9 \
6   --reductions 3 \
7   --depth 6 \
8   --output results/validation.json
9 python experiments/exhaustive_catalog.py \
10  --depth 4 \
11  --moves-per-system 3 \
12  --output results/exhaustive_catalog.json
13
14 cd article
15 ./build.sh

```

The JSON output is deterministic because each generated system uses its integer index as the random seed.

A.3 Claims ledger

The machine-readable file `claims/claims_ledger.yaml` distinguishes:

- proved manuscript claims;
- computational checks and their scope;

- the unexecuted production integration;
- the explicitly unproved general quasi-polynomial goal;
- the absence of an exhaustive literature-priority claim.

The file `claims/research_log.md` records the rejected connected-prefix conjecture, the state-history and inverse-pruning implementation corrections, and the deterministic-policy boundary of the recognition theorem.

A.4 How to audit the key proof mechanically

A lightweight independent audit can proceed as follows.

1. Check that every neutral rule’s guard reads only its declared support and that disjoint supports imply commuting bit toggles.
2. For every unrestricted BFS witness, compute its births, front-load them, replay, and truncate at the first terminal state.
3. Run the birth-front search with the original witness’s birth count and depth. It must find a witness no longer than the oracle witness.
4. Build the support-overlap graph of the oracle witness plus its terminal reduction. If it is disconnected, delete a nonterminal component and obtain a shorter witness, contradicting BFS minimality.

The validation script performs these checks on generated finite systems.

A.5 PDF verification

The compiled PDF should be rendered to images with the repository-independent PDF tooling and inspected for clipped equations, code blocks, table overflow, and broken glyphs. The build in this bundle uses only standard TeX packages and ASCII punctuation in source text where renderer compatibility can be fragile.

A.6 Repository integration boundary

The patch is a review artifact. It should not be merged merely because the abstract tests pass. The repository-side maintainer should:

1. verify the baseline blob or rebase the patch by inspection;
2. add repository-native unit tests for state restoration and replay;
3. run every maintained test and pinned-source check;
4. benchmark failed searches under the same paired methodology used by the synthesis;
5. leave clustered mode opt-in until the evidence supports a default change.

References

- [1] D. Bar–Natan, Khovanov’s homology for tangles and cobordisms, *Geometry & Topology* 9 (2005), 1443–1499.
- [2] I. A. Dynnikov, Arc-presentations of links: monotonic simplification, *Fundamenta Mathematicae* 190 (2006), 29–76; arXiv:math/0208153.
- [3] J. Hass and J. C. Lagarias, The number of Reidemeister moves needed for unknotting, *Journal of the American Mathematical Society* 14 (2001), 399–428; arXiv:math/9807012.
- [4] A. Henrich and L. H. Kauffman, Unknotting unknots, arXiv:1006.4176v4 (2011).
- [5] P. B. Kronheimer and T. S. Mrowka, Khovanov homology is an unknot-detector, *Publications Mathématiques de l’IHES* 113 (2011), 97–208.
- [6] M. Lackenby, A polynomial upper bound on Reidemeister moves, *Annals of Mathematics* 182 (2015), 491–564; arXiv:1302.0180.
- [7] M. Lackenby, Incompressible surfaces, hierarchies and unknot recognition, arXiv:2607.23350v1 (2026).
- [8] OpenAI, Mathematical manuscripts and supporting proof artifacts, <https://github.com/openai/math>, reviewed October 7, 2026.
- [9] V. Reshetnikov, ProveIt: UnknotRecognition, <https://github.com/VladimirReshetnikov/ProveIt/tree/main/Topology/UnknotRecognition>, reviewed at commit 23893144e3aafd074a23cfb633222ab7e03e6d2e, October 7, 2026.